To bind or not to bind? Replaying Object-centric Processes under Stable Relationships

No Author Given

No Institute Given

Abstract. Object-centric process mining investigates the intertwined behavior of multiple objects in business processes. From object-centric event logs, object-centric Petri nets (OCPN) can be discovered to replay the behavior of processes accessing different object types. Although they indicate how objects flow through the process and co-occur in events, OCPNs remain underspecified about the relationships of objects. Hence, they are not able to represent synchronization, i.e. executing objects only according to their intended relationships, and fail to identify violating executions. Existing formal modeling approaches, such as object-centric Petri nets with identifiers (OPID), represent object identities and relationships to synchronize them correctly. However, OPID discovery has not yet been studied. This paper uses explicit data models to bridge the gap between OCPNs and formal OPIDs. We identify the implicit assumptions of stable many-to-one relationships in object-centric event logs, which implies synchronization of related objects. To formally underpin this observation, we combine OCPNs with explicit stable many-to-one relationships in a rigorous mapping from OCPNs to OPIDs explicitly capturing the intended stable relationships and the synchronization of related objects. We prove that the original OCPNs and the resulting OPIDs coincide for those executions that satisfy the intended relationships. Moreover, we provide an implementation of the mapping from OCPN to OPID under stable relationships.

Keywords: Business Process Management · Data Modeling · Object-centric Petri Nets · Conformance Checking · Synchronization.

1 Introduction

Business Process Management aims at modeling and analyzing behavior in information systems [24]. Process mining analyzes event logs that record process executions, which have actually occurred in the system [19]. Process models can be discovered, and the conformance between process models and event logs can be checked [4].

To provide meaningful results, objects that are manipulated during business processes are getting increasing attention recently. In particular, object-centric process mining [17] investigates the interrelation of multiple co-evolving business objects of multiple object types. To record such object-centric processes, different representations are being explored [6], most prominently *object-centric event logs*

(OCEL) [7]. They log which objects are referred to by events and the types of such objects. For example, consider the following OCEL L_1 that represents a behavior recorded in a bicycle manufacturing workshop. For a given order, a frame, a set of wheels, and a handlebar are collected before being assembled:

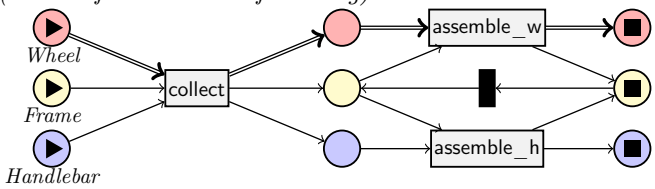
$$\begin{aligned} L_1 = [\ & \langle \mathsf{collect}, \{f_1, h_1, w_1, w_2\}\rangle, \langle \mathsf{assemble_w}, \{f_1, w_1, w_2\}\rangle, \\ & \langle \mathsf{assemble_h}, \{f_1, h_1\}\rangle, \langle \mathsf{collect}, \{f_2, h_2, w_3, w_4\}\rangle, \\ & \langle \mathsf{assemble_h}, \{f_2, h_2\}\rangle, \langle \mathsf{assemble_w}, \{f_2, w_3, w_4\}\rangle \quad] \end{aligned}$$

with f_i of type *frame*, h_i of type *handlebar*, and w_i of type *wheel*. Two bikes are ordered; the first collect event collects the frame, handlebar, and two wheels for the first bike; the `assemble_w` event assembles two wheels to a frame, etc.

To replay the event log and describe the behavior captured in an OCEL, an object-centric Petri net (OCPN) can be discovered [20]. For example, the OCPN in Example 1 allows for replaying the behavior of log L_1. In the initial marking (▶), all recorded objects in the log are placed, so that, e.g., all wheel objects $w_1, \ldots w_4$ reside initially in the *Wheel* place.

The transitions can fire by binding objects and processing them through the net. In OCPNs, a non-variable arc binds a single object, while a variable arc (double line) binds a set of objects. Finally, the net accepts a given execution if all objects are placed in the final marking (■).

Example 1 (OCPN for bike manufacturing).



Although discovered OCPNs consider the token flow of multiple object types, they remain underspecific about their relationships. As a result, OCPNs do not provide mechanisms for tracking the identity of objects, their relationships, and cardinality constraints. This underspecification of discovered OCPNs allows for too much behavior [18]. This becomes apparent if the expected data model with cardinality constraints for log L_1 is explicitly modeled as a UML class diagram.

Example 2 (Data model for bike manufacturing).

The behavior recorded in L_1 satisfies the cardinality constraints for a many-to-one relationship between the wheels and a frame, and a one-to-one relationship between the frame and handlebar. However, the OCPN lacks the semantics for executing objects only in accordance with their relationships, that is, synchronizing them [9], and cannot identify executions that bind unrelated objects! For example, wheels collected for a given frame may be reshuffled, that is, assembled

with a different frame. This is because in OCPNs, the relationships between objects, here, between a frame and its wheels, cannot be represented. For example, the OCPN accepts the execution represented in the following log:

$$L_2 = [\langle \mathsf{collect}, \{f_3, h_3, w_5, w_6\}\rangle, \langle \mathsf{collect}, \{f_4, h_4, w_7\}\rangle, \langle \mathsf{assemble_w}, \{f_3, w_5\}\rangle, \langle \mathsf{assemble_h}, \{f_3, h_4\}\rangle, \langle \mathsf{assemble_w}, \{f_4, w_6, w_7\}\rangle, \langle \mathsf{assemble_h}, \{f_4, h_3\}\rangle]$$

where wheel w_6 is attached to frame f_4 even though it was related to frame f_3 at collection time. This indicates that whenever an OCPN is used in a prescriptive manner for conformance checking or simulation, further so-called "bounding behavior" [18] is needed restricting to only desired synchronizations. This bounding behavior can be suitably captured through more sophisticated object-centric process modeling languages, such as variants of Petri nets with identifiers (PNIDs) [8, 21], or synchronous proclets [5]. Recently, object-centric Petri nets with identifiers (OPIDs [9]) were proposed as a unifying formalism for the features of OCPNs, PNIDs, and synchronous proclets. While OCPNs can only be used as trace recognizers that replay the behavior of recorded objects, OPIDs can also be used as trace generators that prescribe novel executions. However, the discovery of OPIDs and related formalisms has not yet been studied.

This paper bridges the gap between discovered OCPNs and expressive OPIDs by using data models that explicitly define relationships. We identify a crucial but implicit assumption of replaying OCELs: An execution that satisfies a *stable many-to-one relationship* between two object types synchronizes the corresponding objects throughout the execution. This comes from the fact that OCEL assumes global and rigid relationships throughout the entire execution [3, 11]. Whenever an object on the "many" side is bound to a corresponding object on the "one" side, it prevents a different binding at a later point in time, realizing their synchronization.

To formally underpin this observation, this paper provides a rigorous mapping from OCPN combined with a set of stable many-to-one relationships into an OPID implementing the intended synchronization. First, a mapping from an OCPN into an equivalent OPID tracks object identifiers but yields the same ambiguity about relationships. We show their equivalence when replaying OCELs. Second, we consider a set of stable many-to-one relationships to encode the synchronization semantics into the OCPN. The resulting OPID explicitly establishes stable many-to-one relationships and enforces them. We show that the so-obtained OPID and the original OCPN are equivalent in replaying OCELs that satisfy the stable many-to-one relationships, while the OPID prohibits violating executions. Finally, we show that stable many-to-one relationships can be discovered in existing benchmark logs, and we provide an implementation of the mapping.

In the remainder of this paper, Section 2 presents preliminary formalisms. Section 3 defines an equivalent mapping from OCPN to OPID, which is extended by Section 4 to implement stable many-to-one relationships. Section 5 discusses our contribution in the context of related work.

2 Preliminary Definitions

As the basis for our formal investigation, this section summarizes the existing definitions of OCEL and the discovered OCPN [20], as well as OPIDs [9].

2.1 Object-centric event logs and object-centric Petri nets

To define OCELs and OCPNs, we recall notions by van der Aalst and Berti [20].

Definition 1. *Given a set of object types Σ, a set of activities $\mathcal{A}$, and a set of timestamps $\mathbb{T}$ under a total order $<$, an* object-centric event log *(OCEL) is a tuple $L = (E, \mathcal{O}, \pi_{act}, \pi_{obj}, \pi_{time})$ where:*

- *E is a set of events;*
- *$\mathcal{O}$ is a set of object identifiers, each typed by the function $ot : \mathcal{O} \to \Sigma$;*
- *the functions $\pi_{act} : E \to \mathcal{A}$, $\pi_{obj} : E \to \mathcal{P}(\mathcal{O})$, and $\pi_{time} : E \to \mathbb{T}$, mapping each event to an activity, a set of objects, and a timestamp.*

For example, the event logs L_1 and L_2 in Section 1 are defined for the object types *Wheel*, *Frame*, and *Handlebar*. The event $e = \langle \mathsf{collect}, \{f_1, h_1, w_1, w_2\}\rangle$ in L_1 refers to the activity $\pi_{act}(e) = \texttt{collect}$ and the objects $\pi_{obj}(e) = \{f_1, h_1, w_1, w_2\}$, where $ot(f_1) = \mathit{Frame}$.

Definition 2. *Given a set of object types Σ, and a set of activities $\mathcal{A}$, an* object-centric Petri net *(OCPN) is defined as a tuple $N = (P, T, F, pt, F_{var}, \ell)$, where:*

1. *P and T are finite sets of places and transitions such that $P \cap T = \emptyset$,*
2. *$F \subseteq (P \times T) \cup (T \times P)$ is the flow relation between places and transitions,*
3. *$pt\colon P \to \Sigma$ maps every place to an object type,*
4. *$F_{var} \subseteq F$ is the set of variable flow relations,*
5. *$\ell\colon T \to \mathcal{A} \cup \{\tau\}$ is the transition labelling where τ marks an invisible activity.*

The pre-places of a transition t are all places with ingoing flow relations $\bullet t = \{p \in P \mid (p, t) \in F\}$, while the post-places of t are all those connected with outgoing flow relations $t\bullet = \{p \in P \mid (t, p) \in F\}$. For a transition t, the function $tpl(t)$ returns the types of all its corresponding places $p \in \bullet t \cup t\bullet$. $tpl_{var}(t)$ returns all place types connected via variable flow, and $tpl_{nv}(t)$ all place types connected via non-variable flow. An OCPN is considered well-formed if all flow relations of one transition agree in the variability for the place types, i.e., $\forall t \in T : tpl_{var}(t) \cap tpl_{nv}(t) = \emptyset$.

All possible tokens and their location in a place are defined as $Q = \{(p, o) \in P \times \mathcal{O} \mid pt(p) = type(o)\}$. A marking is generally defined as a multiset of token-object pairs $M\colon Q \to \mathbb{N}$ [20]. However, we pose the following specific assumptions about the structure of an OCPN. The contemporary discovery algorithms for OCPN [20, 22] utilize the Inductive Miner algorithm [12], which results in sound workflow nets for each object type. As a result, all places are bound to at most one object identity by design. Therefore, we interpret the marking M of an

OCPN as a set of objects in a marking such that $M\colon Q \to \{0,1\}$. Furthermore, each object type has one initial place and one final place.

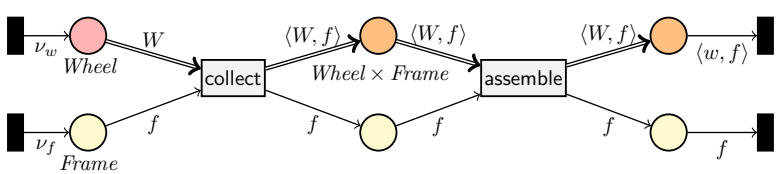
The set of binding executions is $B = \{(t,b) \in T \times (\Sigma \to \mathcal{P}(\mathcal{O})) \mid dom(b) = tpl(t) \wedge \forall \sigma \in tpl_{nv}(t) : |b(\sigma)| = 1\}$. It contains pairs of transition t and binding b. A binding b is a function that maps an object type $\sigma \in \Sigma$ to a set of objects. We denote by $codom(b)$ the codomain of b, that is, $codom(b) = \bigcup_{\sigma \in \Sigma} b(\sigma)$. This binding execution consumes a set of object tokens from the transition's pre-places $cons(t,b) = \{(p,o) \in Q \mid p \in \bullet t \wedge o \in b(pt(p))\}$. Similarly, it produces object tokens $prod(t,b) = \{(p,o) \in Q \mid p \in t\bullet \wedge o \in b(pt(p))\}$. This binding is enabled in a marking M_i and can be executed if $cons(t,b) \leq M_i$. The resulting marking is $M_j = M_i - cons(t,b) + prod(t,b)$. The firing is denoted as $M_i \xrightarrow{(t,b)} M_j$. Note that by the definition of bindings, if a transition t has two places in $p_1, p_2 \in \bullet t$ of the same object type, then t can only be fired by consuming the same objects from p_1 and p_2. A binding (t,b) for a transition t considers only objects $b \subseteq \mathcal{O}$ that belong to the types of the respective input places and are in line with the variable and non-variable arcs.

An *accepting OCPN* combines an OCPN N with an initial marking $M_{init} \in \mathcal{B}(Q)$ and a final marking $M_{final} \in \mathcal{B}(Q)$. An accepted run is a sequence of binding executions from initial to final marking $M_{init} \xrightarrow{*} M_{final}$

The accepting OCPN can be used to *replay* the behavior of the log. All recorded objects are interpreted as the initial marking M_{init}. An event in the log and its recorded objects are interpreted as a binding execution of the corresponding transition and the involved objects, which results in a new marking. After replaying all events consecutively, the OCPN can accept the log if the run results in the final marking M_{final}. If an event has no corresponding binding execution or M_{final} is not reached, the log is rejected by the OCPN.

2.2 Object-centric Petri nets with identifiers

Example 3 (Simple OPID for bike manufacturing). The OPID combines the object types *Wheel* (red), *Frame* (yellow), and relations for both (orange places).



We summarize the definitions from [9]. Every object type $\sigma \in \Sigma$ is assumed to have a domain $dom(\sigma) \subseteq \mathcal{O}$, given by all objects in $\mathcal{O}$ of type σ. List types with a base type in σ are denoted as $[\sigma]$. Each Petri net place has a *color* that is a cartesian product of data types from Σ; precisely, the set of colors $\mathcal{C}$ is the set of all $\sigma_1 \times \cdots \times \sigma_m$ such that $m \geq 1$ and $\sigma_i \in \Sigma$ for all $1 \leq i \leq m$. We fix a set of Σ-typed variables $\mathcal{X} = X \uplus X_{list} \uplus \Upsilon$ as the disjoint union of a set X of "normal" variables that refer to single objects, denoted by lower-case letters like x, with a type $\text{type}(x) \in \Sigma$, a set X_{list} of list variables that refer to a list

of objects of the same type, denoted by upper case letters like U, with a type $\text{type}(U) = [\sigma]$ for some $\sigma \in \Sigma$, and a set Υ of variables referring to fresh objects, denoted as ν, with $\text{type}(\nu) \in \Sigma$.

In OPIDs [9], tokens are object tuples associated with a color. To define relationships between objects in consumed and produced tokens when firing a transition, arc *inscriptions* are defined.

Definition 3. *An* inscription *is a tuple* $\boldsymbol{x} = \langle x_1, \ldots, x_m \rangle$ *such that* $m \geq 1$ *and* $x_i \in \mathcal{X}$ *for all* i*, but at most one* $x_i \in X_{list}$*, for* $1 \leq i \leq m$*. We call* $\boldsymbol{x}$ *a* template inscription *if* $x_i \in X_{list}$ *for some* i*, and a* simple inscription *otherwise.*

Template inscriptions capture an arbitrary number of tokens of the same color; they are used to label the correspondent of variable arcs in OCPNs. The color of an inscription $\iota = \langle x_1, \ldots, x_m \rangle$ is given by $\text{color}(\iota) = \langle \sigma_1, \ldots, \sigma_m \rangle$ where $\sigma_i = \text{type}(x_i)$ if $x_i \in X \cup \Upsilon$, and $\sigma_i = \sigma'$ if x_i is a list variable of type $[\sigma']$. We set $\text{vars}(\iota) = \{x_1, \ldots, x_m\}$. The set of all inscriptions is denoted Ω. Based on this, an OPID is formally defined as follows [9].

Definition 4. *An* object-centric Petri net with identifiers *(OPID) is defined as a tuple* $N = (\Sigma, P, T, F_{in}, F_{out}, \text{color}, \ell)$*, where:*

1. P *and* T *are finite sets of places and transitions such that* $P \cap T = \emptyset$*,*
2. $\text{color}\colon P \to \mathcal{C}$ *maps every place to a color,*
3. $\ell\colon T \to \mathcal{A} \cup \{\tau\}$ *is the transition labelling where* τ *marks an invisible activity,*
4. $F_{in}\colon P \times T \to \Omega$ *is a partial function called* input flow*, such that* $\text{color}(F_{in}(p,t)) = \text{color}(p)$ *for every* $(p,t) \in dom(F_{in})$*,*
5. $F_{out}\colon T \times P \to \Omega$ *is a partial function called* output flow*, such that* $\text{color}(F_{out}(t,p)) = \text{color}(p)$ *for every* $(t,p) \in dom(F_{out})$*.*

We set $\text{vars}_{in}(t) = \bigcup_{p \in P} \text{vars}(F_{in}(p,t))$*, and* $\text{vars}_{out}(t) = \bigcup_{p \in P} \text{vars}(F_{out}(t,p))$*, and require that* $\text{vars}_{in}(t) \cap \Upsilon = \emptyset$ *and* $\text{vars}_{out}(t) \subseteq \text{vars}_{in}(t) \cup \Upsilon$*, for all* $t \in T$*.*

If $F_{in}(p,t)$ is defined, it is called a *variable* flow if $F_{in}(p,t)$ is a template inscription, and *non-variable* flow otherwise; and similar for output flows. Variable flows play the role of variable arcs in OCPNs [20]; they can carry multiple tokens at once. The common notations $\bullet t = \{p \mid (p,t) \in dom(F_{in})\}$ and $t\bullet = \{p \mid (t,p) \in dom(F_{out})\}$ are used as well.

Semantics. Given the set of objects $\mathcal{O}$, the set of *tokens* $\boldsymbol{\mathcal{O}}$ is the set of object tuples $\boldsymbol{\mathcal{O}} = \{\mathcal{O}^m \mid m \geq 1\}$. The *color* of a token $\omega \in \boldsymbol{\mathcal{O}}$ of the form $\omega = \langle o_1, \ldots, o_m \rangle$ is denoted $\text{color}(\omega) = \langle \text{type}(o_1), \ldots, \text{type}(o_m) \rangle$. A *marking* of an OPID $N = \langle \Sigma, P, T, F_{in}, F_{out}, \text{color}, \ell \rangle$ is a function $M\colon P \to 2^{\boldsymbol{\mathcal{O}}}$, such that for all $p \in P$ and $\langle o_1, \ldots o_m \rangle \in M(p)$, it holds that $\text{color}(\langle o_1, \ldots o_m \rangle) = \text{color}(p)$. Let $Lists(\mathcal{O})$ denote the set of objects lists of the form $[o_1, \ldots, o_k]$ with $o_1, \ldots, o_k \in \mathcal{O}$ such that all o_i have the same type; the type of such a list is then $[\text{type}(o_1)]$. Next, *bindings* are defined to fix which objects are involved in a transition firing.

Definition 5. *A* binding *for a transition* t *and a marking* M *is a type-preserving function* $\beta\colon \text{vars}_{in}(t) \cup \text{vars}_{out}(t) \to \mathcal{O} \cup Lists(\mathcal{O})$*. To ensure the freshness of created values,* β *is required to be injective on* $\Upsilon \cap \text{vars}_{out}(t)$*, and* $\beta(\nu)$ *must not occur in* M *for all* $\nu \in \Upsilon \cap \text{vars}_{out}(t)$*.*

We denote by $codom(\beta) \subseteq \mathcal{O}$ the set of all objects occurring in tuples returned by β. Bindings are extended to inscriptions, denoted $\boldsymbol{\beta}$, to fix which tokens participate in a transition firing. For an inscription $\iota = \langle x_1, \ldots, x_m \rangle$, let $o_i = \beta(x_i)$ for all $1 \leq i \leq m$. Then $\boldsymbol{\beta}(\iota)$ is the set of object tuples defined as follows: if ι is a simple inscription, then $\boldsymbol{\beta}(\iota) = \{\langle o_1, \ldots, o_m \rangle\}$. Otherwise, there must be one x_i, $1 \leq i \leq n$, such that $x_i \in X_{list}$, and consequently o_i must be a list, say $o_i = [u_1, \ldots, u_k]$ for some $u_1, \ldots, u_k$. Then $\boldsymbol{\beta}(\iota) = \{\langle o_1, \ldots, o_{i-1}, u_1, o_{i+1}, \ldots, o_m \rangle, \ldots, \langle o_1, \ldots, o_{i-1}, u_k, o_{i+1}, \ldots, o_m \rangle\}$. $\mathcal{B}$ denotes the set of all bindings. Finally, a transition with a binding β is enabled if all object tuples pointed by $\boldsymbol{\beta}$ occur in the current marking:

Definition 6. *A transition $t \in T$ and a binding β for marking M are* enabled *in M if $\boldsymbol{\beta}(F_{in}(p,t)) \subseteq M(p)$ for all $p \in \bullet t$.*

Definition 7. *Let transition t be enabled in marking M with binding β. The* firing *of t yields the new marking M' given by $M'(p) = M(p) \setminus \boldsymbol{\beta}(F_{in}(p,t))$ for all $p \in \bullet t \setminus t\bullet$, and $M'(p) = M(p) \cup \boldsymbol{\beta}(F_{out}(p,t))$ for all $p \in t\bullet \setminus \bullet t$, and $M'(p) = M(p) \setminus \boldsymbol{\beta}(F_{in}(p,t)) \cup \boldsymbol{\beta}(F_{out}(p,t))$ for all $p \in \bullet t \cap t\bullet$.*[1]

We write $M \xrightarrow{t,\beta} M'$ to denote that t is enabled for binding β in M, and its firing yields M'. An *accepting* OPID N is an OPID together with a set of initial markings M_{init} and a set of final markings M_{final}. A run is accepted if it starts in the initial and ends in the final marking $M_{init} \xrightarrow{*} M_{final}$.

3 Equivalent Object-centric Petri Nets with Identifiers

This section defines the mapping $\mathbb{T}_1$ from an OCPN N to an OPID. We show that the replays of both nets N and $\mathbb{T}_1(N)$ on a given event log are equivalent, that is, they accept and reject the same executions.

In this paper, we assume that for any given OCPN $N = (P, T, F, pt, F_{var}, \ell)$ there are fixed sets of places $P_{\blacktriangleright}, P_{\blacksquare} \subseteq P$ called *play* and *stop places*, respectively, such that for each $\sigma \in \Sigma$ there is exactly one place $p \in P_{\blacktriangleright}$ and $p' \in P_{\blacksquare}$ such that $pt(p) = pt(p') = \sigma$. These are the places that are supposed to be populated by initial and final markings, respectively. Markings M_{init} and M_{final} are *suitable* initial and final markings for an OCPN with play and stop places $P_{\blacktriangleright}, P_{\blacksquare}$ if they put tokens only in those places, i.e., for all $o \in \mathcal{O}$, $M_{init}(p, o) = 0$ for all $p \notin P_{\blacktriangleright}$ and $M_{init}(p', o) = 0$ for all $p' \notin P_{\blacksquare}$, and similar for M_{final}.

Note that a given set of objects uniquely determines suitable initial and final markings for N: precisely, for a finite set of objects $O \subseteq \mathcal{O}$, let M^O_{init} (resp. M^O_{final}) be the marking such that for each $\sigma \in \Sigma$ and and each $o \in O_\sigma$, $M^O_{init}(p_\sigma, o) = 1$ (resp. $M^O_{final}(p_\sigma, o) = 1$) where p_σ is the unique place in $P_{\blacktriangleright}$ (resp. $P_{\blacksquare}$) of type σ. Our mapping $\mathbb{T}_1$ maps an OCPN N with start and stop places to an OPID $\mathbb{T}_1(N)$ that keeps all the places and transitions of N. The flow relations are adjusted and extended with arc inscriptions. Moreover, $\mathbb{T}_1(N)$ has additional

[1] The case for $p \in \bullet t \cap t\bullet$ was omitted in [9], which is insufficient for cyclic nets.

transitions for creating and removing objects. In this way, the OPID can, on demand, populate the places in $P_{\blacktriangleright}$ with objects in an initial marking for N and consume objects from the places in $P_{\blacksquare}$ once a final marking for N is reached. The initial and final markings for $\mathbb{T}_1(N)$ are always empty.

Precisely, the OPID has an emitting transition for every object type, i.e., we set $T_e := \{t_{e,\sigma} \mid \sigma \in \Sigma\}$; and a consuming transition for every object type, i.e., we set $T_c := \{t_{c,\sigma} \mid \sigma \in \Sigma\}$. With its emitting transitions T_e, an OPID allows the creation of arbitrary new objects. However, in the replay of an OCEL L, this set of objects is fixed and denoted as $\mathcal{O}_L$. Hence, when replaying the log L, the inscription variables $\nu_i \in F_{out}(T_e)$ will be bound exclusively to objects in $\mathcal{O}_L$. To be able to define suitable inscriptions, we assume that for every object type $\sigma \in \Sigma$ there is a normal variable $x_\sigma \in X$, a list variable $X_\sigma \in X_{list}$, and a fresh object variable $\nu_\sigma \in \Upsilon$. We then set:

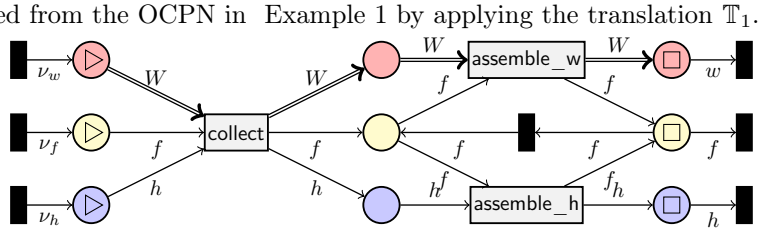
$$F_{out} \colon (T \cup T_e) \times P \to \Omega \qquad F_{out}(t,p) = \begin{cases} \langle x_{pt(p)} \rangle & \text{if } (t,p) \in F \setminus F_{var} \\ \langle X_{pt(p)} \rangle & \text{if } (t,p) \in F_{var} \\ \langle \nu_{pt(p)} \rangle & \text{if } p \in P_{\blacktriangleright} \wedge t \in T_e \end{cases}$$

$$F_{in} \colon P \times (T \cup T_c) \to \Omega \qquad F_{in}(p,t) = \begin{cases} \langle x_{pt(p)} \rangle & \text{if } (p,t) \in F \setminus F_{var} \\ \langle X_{pt(p)} \rangle & \text{if } (p,t) \in F_{var} \\ \langle x_{pt(p)} \rangle, & \text{if } p \in P_{\blacksquare} \wedge t \in T_c \end{cases}$$

The equivalent OPID is given by $\mathbb{T}_1(N) = (\Sigma_1, P_1, T_1, F_{in}, F_{out}, color, \ell_1)$ where $\Sigma_1 = \Sigma$, $P_1 = P$, $T_1 = T \cup T_e \cup T_c$, $color(p) = \langle pt(p) \rangle$ for all $p \in P$, and

$$\ell_1 \colon T \to \mathcal{A} \qquad \ell_1(t) = \begin{cases} \tau, & \text{if } t \in T_e \cup T_c \\ \ell(t), & \text{otherwise} \end{cases}$$

Example 4 (Equivalent OPID for bike manufacturing). The following OPID is obtained from the OCPN in Example 1 by applying the translation $\mathbb{T}_1$.



$\mathbb{T}_1(N)$ is interpreted as an accepting OPID where the initial and final markings are defined as the empty marking $M_\emptyset$, which requires that all places be empty. The example OPID can replay the logs L_1 as well as L_2 as it has equivalent semantics to Example 1 in combining objects without additional constraints.

As formalized in Theorem 1, the execution semantics of the OCPN N and the mapped OPID $\mathbb{T}_1(N)$ are equivalent for replaying any OCEL. This theorem can be proven by considering the equivalent definition of binding executions. Considering $M_i' = \mathbb{T}_1(M_i)$ as the mapped markings and $\beta_i = \mathbb{T}_1(b_i)$ as the

mapped bindings, $\mathbb{T}_1(N)$ can produce and consume the same set of objects $\mathcal{O}$ as specified in the initial and final marking of the OCPN. For reasons of space, the full proof of the following Theorem 1 is moved to Appendix A of the extended version of this paper.[2]

Theorem 1. *Let $\mathcal{O}$ be a set of objects, N be an accepting OCPN, $N' = \mathbb{T}_1(N)$, and $M_\emptyset$ denote the empty marking of N'. There exists a run of N:*

$$M_0 \xrightarrow{(t_1,b_1)} M_1 \xrightarrow{(t_2,b_2)} \cdots \xrightarrow{(t_n,b_n)} M_n$$

with $M_0{=}M^{\mathcal{O}}_{init}$ and $M_n{=}M^{\mathcal{O}}_{final}$ if and only if there is a run of N' over the set of objects $\mathcal{O}$:

$$M_\emptyset \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset$$

such that $M'_i = \mathbb{T}_1(M_i)$ for all $0 \leq i \leq n$ and $\beta_i = \mathbb{T}_1(b_i)$ for all $1 \leq i \leq n$, such that $M_\emptyset \xrightarrow{*} M'_0$ is an emitting sequence, $M'_n \xrightarrow{*} M_\emptyset$ a collapsing sequence, and no emitting or consuming transitions occur in between.

4 Synchronization for Stable Many-to-One Relationships

In this section, we define what constitutes a stable many-to-one relationship for object-centric process executions. Given OCEL that assumes the stability of relationships, such a stable many-to-one relationship implies that objects on the many-side will always be bound to the same object of the one-side. Hence, the processing of such objects is implicitly synchronized. We then extend the mapping $\mathbb{T}_1$ from Section 3 to implement the expected synchronization for a set of defined stable many-to-one relationships. Finally, we show that any execution replay of the OCPN is accepted by the mapped OPID if and only if it conforms to the stable many-to-one relationships.

In the following, let a pair (σ_m, σ_o) be an *object type relationship* if $\sigma_m \neq \sigma_o$. For example, for the bike manufacturing example, a stable many-to-one relationship may exist for *(Wheel, Frame)*. Given an OCEL $L = (E, \mathcal{O}, \pi_{act}, \pi_{obj}, \pi_{time})$, we define the function $lo\colon \mathcal{O} \times \Sigma \to \mathcal{O}$, which, given an object and an object type, returns all co-occurring objects of this type, i.e. $lo(o, \sigma) = \bigcup_{e \in E}\{o' \mid o, o' \in \pi_{obj}(e) \wedge ot(o') = \sigma\}$. For OCPNs and OPIDs, lo is defined accordingly over a run of the net and bindings.

Definition 8. *An object type relationship (σ_m, σ_o) is a* stable many-to-one relationship *in an event log L if for all objects $o_m, o_o \in \mathcal{O}$, where $ot(o_m) = \sigma_m$ and $ot(o_o) = \sigma_o$ it holds that $|lo(o_m, \sigma_o)| = 1$.*

Similarly for OCPNs and OPIDs, (σ_m, σ_o) is a *stable many-to-one relationship* if the same holds for a run in the net: The objects o_m of the many-side σ_m in the stable relationship (σ_m, σ_o) are required to be linked to exactly one

[2] `https://anonymous.4open.science/r/To-bind-or-not-to-bind-43DD`

object o_o of the one-side σ_o. On the other hand, every o_o must have a non-empty set of linked objects o_m. For example, *(Wheel, Frame)* is a stable many-to-one relationship in the log L_1 from Section 1, but not in L_2. Note that the one-to-one relationship between handlebar and frame manifests itself as two stable many-to-one relationships *(Handlebar, Frame)* and *(Handlebar, Frame)*.

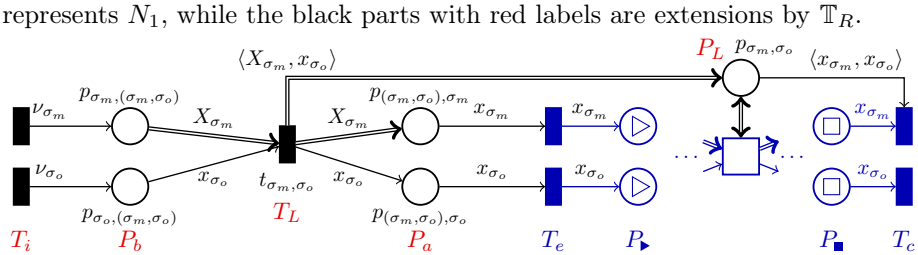
4.1 Mapping to synchronizing OPIDs

The goal of this mapping is to enforce a set of stable many-to-one relationships with the synchronization behavior of an OPID. To this end, for every many-to-one relationship (σ_m, σ_o), the required set of links $(m, o) \in \mathcal{O}^2$ of type (σ_m, σ_o) must be determined *before* the actual execution starts, and must be stable during the whole execution. In general, the idea is to store all links of a relationship in a dedicated place that is accessed by all transitions that operate on both object types. For the Petri net used for the manufacturing of bikes, the translation results in the OPID shown in Example 5. For the log L_1, it determines the links between wheels w_1, w_2 of type W and frame f_1 of type F as $(w_1, f_1), (w_2, f_1)$ before execution starts and they are not changed. The transition's synchronization semantics ensure that only objects in a stable relationship are within one binding.

We assume a given OCPN $N = (P, T, F, pt, F_{var}, \ell)$ and a fixed set of object type relationships R, which are supposed to work as stable many-to-one relationships. The mapping $\mathbb{T}_R$ is defined on top of the mapping $\mathbb{T}_1$ in Section 3. Let $N_1 = \mathbb{T}_1(N) = (\Sigma_1, P_1, T_1, F_{in,1}, F_{out,1}, \text{color}_1, \ell_1)$. In $\mathbb{T}_R(N_1)$, we add the following new places and transitions:

- a set of *link places* $P_L := \{p_{(\sigma_m,\sigma_o)} \mid (\sigma_m, \sigma_o) \in R\}$, storing the links for each relationship $(\sigma_m, \sigma_o) \in R$, such that $\text{color}_2(p_{(\sigma_m,\sigma_o)}) = \langle \sigma_m, \sigma_o \rangle$,
- a set of *link transitions* $T_L := \{t_{\sigma_m,\sigma_o} \mid (\sigma_m, \sigma_o) \in R\}$ that creates links for a set of objects of type σ_m and one object of σ_o to populate P_L,
- new emitting transitions $T_i := \{t_{i,\sigma} \mid \exists \sigma' \in \Sigma : (\sigma, \sigma') \in R \vee (\sigma', \sigma) \in R\}$,
- a set $P_b := \{p_{\sigma,(\sigma_m,\sigma_o)} \mid (\sigma = \sigma_m \vee \sigma = \sigma_o) \wedge (\sigma_m, \sigma_o) \in R\}$ of auxiliary places before the link creation such that $\text{color}_2(p_{\sigma,(\sigma_m,\sigma_o)}) = \langle \sigma \rangle$, and
- a set $P_a := \{p_{(\sigma_m,\sigma_o),\sigma} \mid (\sigma = \sigma_m \vee \sigma = \sigma_o) \wedge (\sigma_m, \sigma_o) \in R\}$ of auxiliary places after link creation such that $\text{color}_2(p_{\sigma,(\sigma_m,\sigma_o)}) = \langle \sigma \rangle$.

The arrangement of the additional places and transitions is schematically shown below for a pair of object types $(\sigma_m, \sigma_o) \in R$. Here, the blue part of the net represents N_1, while the black parts with red labels are extensions by $\mathbb{T}_R$.



Before defining the new flow relations, we give an intuitive explanation. The idea is that all links are created concurrently before the objects are placed in

the initial places $P_{\blacktriangleright}$ by T_e. To that end, new emitting transitions T_i are used to create objects which are placed in P_b. Every link creation transition $t_{\sigma_m,\sigma_o} \in T_L$ is connected with inflow to the according places from P_b, and with outflow to the respective link place $p_{\sigma_m,\sigma_o} \in P_L$ as well as places in P_a. The places in P_a are connected to the former emitting transitions T_e. When all transitions T_e emitted the objects to the places $P_{\blacktriangleright}$, the places in P_L are thus populated with a fixed set of links. Crucially, to enforce the synchronization for a transition t in N_1, which operates on σ_m and σ_o, the respective transition in $\mathbb{T}_R(N_1)$ must read and write the required links from the link places P_L. Finally, a link is consumed together with its object by the consuming transition in T_c.

Precisely, we define the flow relations as follows.

- For a new emitting transition $t_{i,\sigma} \in T_i$, and $p \in P_b$ where $\text{color}_2(p) = \langle\sigma\rangle$, the outflow is set as $F_{out,2}(t_{i,\sigma}, p) = \langle\nu_\sigma\rangle$.
- For a former emitting transition $t_{e,\sigma} \in T_e$ and $p \in P_{\blacktriangleright}$ with $\text{color}_2(p) = \langle\sigma\rangle$, we set $F_{out,2}(t_{e,\sigma}, p) = \langle x_\sigma\rangle$ if there is a $\sigma' \in \Sigma$ such that (σ, σ') or (σ', σ) in R, otherwise $F_{out,2}(t_{e,\sigma}, p) = \langle\nu_\sigma\rangle$ as before.
- For a place in P_b which is for the one-side of the relationship, the inflow reads a single object $F_{in,2}(p_{\sigma_o,(\sigma_m,\sigma_o)}, t_{\sigma_m,\sigma_o}) = \langle x_{\sigma_o}\rangle$; while for the many-side we have $F_{in,2}(p_{\sigma_m,(\sigma_m,\sigma_o)}, t_{\sigma_m,\sigma_o}) = \langle X_{\sigma_m}\rangle$ reading a list of objects.
- Similarly, for a place in P_a for the one-side of a relationship we have $F_{out,2}(t_{\sigma_m,\sigma_o}, p_{(\sigma_m,\sigma_o),\sigma_o}) = \langle x_{\sigma_o}\rangle$ whereas for a place for the many-side we set $F_{out,2}(t_{\sigma_m,\sigma_o}, p_{(\sigma_m,\sigma_o),\sigma_m}) = \langle X_{\sigma_m}\rangle$.
- Each link creation transition $t_{\sigma_m,\sigma_o} \in T_L$ has an outflow to the respective place $p_{(\sigma_m,\sigma_o)} \in P_L$ with inscription $F_{out,2}(t_{\sigma_m,\sigma_o}, p_{(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m}, x_{\sigma_o}\rangle$.
- Every former emitting transition $t_{e,\sigma} \in T_e$ has an inflow from all related $p \in P_a$ where $\text{color}_2(p) = \langle\sigma\rangle$, so that $F_{in,2}(p, t_{e,\sigma}) = \langle x_\sigma\rangle$.
- For all link places $p_{(\sigma_m,\sigma_o)} \in P_L$ and transitions $t \in T$ where there exist places $p_1, p_2 \in P$ such that $\text{color}_2(p_1) = \langle\sigma_m\rangle$ and $\text{color}_2(p_2) = \langle\sigma_o\rangle$ and both $F_{in,1}(p_1, t)$ and $F_{in,1}(p_2, t)$ are defined, we set $F_{in,2}(p_{(\sigma_m,\sigma_o)}, t) = F_{out,2}(t, p_{(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m}, x_{\sigma_o}\rangle$.
- Finally, for all link places $p_{(\sigma_m,\sigma_o)} \in P_L$, an arc to the consuming transition $t_{c,\sigma_m} \in T_c$ consumes the links $F_{in,2}(p_{(\sigma_m,\sigma_o)}, t_{c,\sigma_m}) = \langle x_{\sigma_m}, x_{\sigma_o}\rangle$.

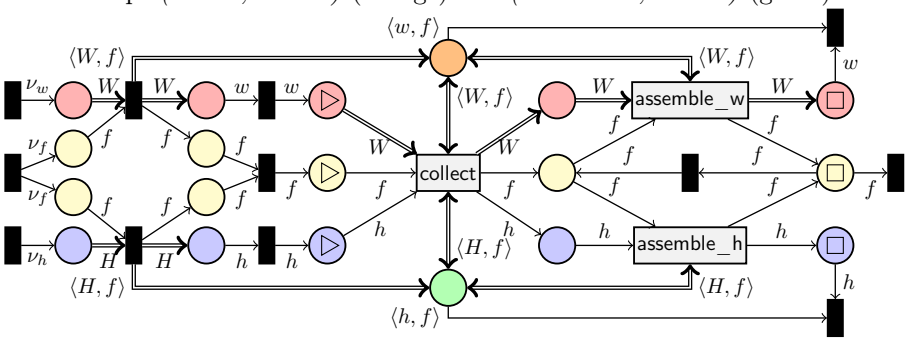
Note that for the modification of transitions from the OCPN, we assume that the OCPN is constructed from sound workflow nets. Hence, every transition has an inflow for an object type if and only if it has an outflow for it. For example, if $p_{W,F}$ holds the tokens $(w_1, f_1), (w_2, f_1)$, then the transition `assemble_w` can only bind the frame f_1 to any of the wheels w_1 or w_2 but never to w_3.

For all other $p \in P_1$ and $t \in T_1$ we set $F_{in,2}(p, t) = F_{in,1}(p, t)$, $F_{out,2}(t, p) = F_{out,1}(t, p)$, and $\text{color}_2(p) = \text{color}_1(p)$. The new places are $P_2 = P_1 \cup P_b \cup P_L \cup P_a$, and the new transitions are $T_2 = T_1 \cup T_i \cup T_L$. Then, the result of the mapping $\mathbb{T}_R$ is the OPID $\mathbb{T}_R(\mathbb{T}_1(N)) = (\Sigma_1, P_2, T_2, F_{in,2}, F_{out,2}, \text{color}_2, \ell_2)$ where $\ell_2(t) = \tau$ for all $t \in T_i \cup T_L$ and $\ell_2(t) = \ell_1(t)$ otherwise.

The OPID N_2 is again interpreted as an accepting OPID where the initial marking $M_{init,2}$ and the final marking $M_{final,2}$ require that all places be empty. The OPID in Example 5 accepts L_1 because there exists a run, where the links for

wheels and frames can be set as $(w_1, f_1), (w_2, f_1), (w_3, f_2), (w_4, f_2)$ in the orange link place and the tokens for handlebars and frames as $(h_1, f_1), (h_2, f_2)$ in the green link place. The transitions bind objects correctly according to the links. However, the log-run L_2 is rejected because $\langle \mathsf{collect}, \{f_3, h_3, w_5, w_6\}\rangle$ requires the links (w_5, f_3) and (w_6, f_3) but $\langle \mathsf{assemble_w}, \{f_4, w_6, w_7\}\rangle$ requires the link (w_6, f_4), which cannot exist at the same time.

Example 5 (Synchronizing OPID for bike manufacturing). The OPID is obtained from the OCPN in Example 1 by applying $\mathbb{T}_1$ and $\mathbb{T}_R$ for the stable relationships *(Wheel, Frame)* (orange) and *(Handlebar, Frame)* (green).



4.2 Equivalence for executions with stable relationships

In Section 3, we show that an OCPN N is equivalent to its OPID representation $N_1 = \mathbb{T}_1(N)$ for event log replay. The mapping $N_2 = \mathbb{T}_R(N_1)$ implements the stable relationships R into N_1. We show that a run in the OCPN N is conform to R if and only if there is an accepted corresponding run in N_2.

Theorem 2. *Let $N_1 = \mathbb{T}_1(N)$ be an equivalent OPID for an OCPN N and $N_2 = \mathbb{T}_R(N_1)$ a synchronizing OPID. There is an accepted run of N_1 of the form*

$$M_\emptyset \xrightarrow{*} M_0 \xrightarrow{(t_1,\beta_1)} M_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M_n \xrightarrow{*} M_\emptyset$$

that conforms to the stable relationships R if and only if there is an accepted run of N_2 of the form

$$M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset$$

such that $\forall p \in P_1 \colon M'_i(p) = M_i(p)$ for all $0 \leq i \leq n$.

While the detailed proof for Theorem 2 is spelled out in Appendix B of the extended version of the paper[3], we outline its three main arguments: (i) Any accepted run in N_2 is accepted in N_1. (ii) Any accepted run in N_1 that conforms to R is accepted by N_2. (iii) Any accepted run in N_1 that not conforming to R is rejected by N_2.

[3] `https://anonymous.4open.science/r/To-bind-or-not-to-bind-43DD`

(i) The emitting and link creating sequence $M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0$ results in a marking M'_0 that holds only tokens in the places $P_\blacktriangleright \cup P_L$. An equivalent emitting sequence $M_\emptyset \xrightarrow{*} M_0$ exists for N_1 that produces the same objects in the initial places $P_\blacktriangleright$. Any binding execution (t_i, β_i) in the run of N_2 can be replayed in a corresponding marking of N_1 as it is less restrictive. Finally, there exists an appropriate sequence $M_n \xrightarrow{*} M_\emptyset$ of only consuming transitions $t \in T_c$ that removes the same objects as $M'_n \xrightarrow{*} M_\emptyset$.

(ii) For the conforming run $M_\emptyset \xrightarrow{*} M_0$ that places objects to $P_\blacktriangleright$ in N_1, there exists a corresponding run $M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0$ in N_2, which places the same objects onto the places P_b in M'_b. Then, $M'_b \xrightarrow{*} M'_a$ creates all required links according to the binding executions (t_i, β_i) in the run of N_1, such that for every object o_m of type σ_m in a relationship with an object of type σ_o, the links in P_L are set for all $lo(o_m, \sigma_o)$. Next, $M'_a \xrightarrow{*} M'_0$ places the objects in $P_\blacktriangleright$ such that M'_0 holds tokens only in places $p' \in P_\blacktriangleright \cup P_L$. The created links are not changed and transitions that operate on two object types in a relationship are required to adhere to them.
If the run in N_1 is conform, it only binds an object o_m to a single object of σ_o, hence, $|lo(o_m, \sigma_o)| = 1$ and there exists a link such that N_2 can accept the run as well. Finally, for $M_n \xrightarrow{*} M_\emptyset$ of only consuming transitions $t \in T_c$, $M'_n \xrightarrow{*} M_\emptyset$ can consume the same objects including their respective links.

(iii) By contradiction, we show that N_2 cannot create links that violate the conformance criteria. No object in a relationship can exist without a link. Any object o_m with $ot(o_m) = \sigma_m$ will receive exactly one link to an object of type σ_o and not more. As the bindings of any run in N_2 are restricted by the links, a non-conforming run of N_1 has no counterpart in N_2.

By expanding the mapping $\mathbb{T}_1$ with stable relationships in $\mathbb{T}_R$ for a given OCPN and its relationships R, we can generate an OPID $\mathbb{T}_R(\mathbb{T}_1(N))$ that accepts an execution in N if and only if it conforms to R.

5 Discussion and Related Work

In the following, we first investigate the presence of stable many-to-one relationships in existing benchmark OCELs and their discussion in related work. Second, we describe a prototypical implementation of our mapping. Finally, we comment on the implications for the research areas of conformance checking and formal analysis.

5.1 Stable Relationships in OCEL

Stable relationships can be discovered at the type level from OCELs, as described in the TOTeM tool [14]. To substantiate our intuition that, in many contexts, the executions of object-centric processes indeed have stable relationships, we applied an according analysis to existing benchmark logs[4] The result in Table 1 shows that all investigated logs contain some stable many-to-one relationships.

[4] `https://anonymous.4open.science/r/To-bind-or-not-to-bind-43DD`

Table 1. OCELs with their number of object types, all relationships (including one-to-one), stable many-to-one relationships with, and without filtering noise.

OCEL	#obj. types	#relationships	#m2o(noise=0.1)	#m2o(noise=0.0)
Logistics	7	9	6 (4+2*1)	6 (4+2*1)
Order Management	6	13	5 (5+0)	5 (5+0)
Procure to Pay	7	9	10 (6+2*2)	10 (6+2*2)
Hinge Production	12	39	35 (13+2*11)	35 (13+2*11)
Age of Empires	30	135	**87** (77+2*5)	**67** (57+2*5)
LRMS – O2C	9	11	**18** (2+2*8)	**16** (4+2*6)
LRMS – P2P	6	15	**11** (3+2*4)	**0** (0+0)
LRMS – Hiring	6	15	6 (6+0)	6 (6+0)
LRMS – Hospital	8	18	6 (6+0)	6 (6+0)

The stability of object relationships is a typical (often implicit) assumption in the literature. In the widely adopted OCEL 2.0 standard [3], links are recorded without a notion of time, assuming global existence and stability. For discovered models, such as OCPNs [20], class diagrams [14], and even simulation models [11], this yields the same notion of stable relationship introduced here.

The intended stable relationships may be violated in OCELs. When investigating noise, that is, infrequent outliers, it can be seen that some logs contain a few violations of the intended stable many-to-one relationships. In Table 1, we report how the number of stable relationships changes when considering noise. For example, LLM-based simulations for LRMS – O2C contain 20 more stable relationships when only focusing on the 90% most frequent observations. This relates to the approach in [26], where this type of violation is identified in a real-world use case. Our mapping to OPIDs can assist in identifying exact executions that violate stable relationships.

However, our identified assumptions yield some limitations. It should be noted that our definition of stable many-to-one relationships only considers existential relations of the type `(1..*,1..1)`. Non-existential relationships (e.g. `(0..*,0..1)`) are not present in the existing OCELs but should be considered in future investigations as well. In general, the need to enrich object-centric event data with concepts from data modeling such as non-rigid relationships is being increasingly recognized. Goossens et al. [10] point out that from a database perspective, links between objects might indeed change over time. It is not possible to differentiate between a non-rigid many-to-one relationship and an intended many-to-many relationship just by looking at the OCEL without domain knowledge. Therefore, our approach emphasizes an explicit declaration of object types and relationships, e.g., as data models. In addition, [16] highlights the need for richer representations of object-centric event data, where the effects of events are explicitly tracked, allowing identification of time intervals within which a relationship exists. An ongoing effort aims to consolidate these different considerations in a single standard for representing object-centric event data [6], motivating further research on the matter.

5.2 Process Discovery and Prototypical Implementation

OCPNs can be easily discovered through the Python libraries provided [1]. We provide a prototypical implementation[5] of our proposed mapping to produce OPIDs, which builds on existing libraries and uses their resulting OCPNs as input. Such an OCPN can be combined with a set of intended stable many-to-one relationships to produce the corresponding OPID. The provided repository contains a set of mapped OPIDs to show the technical feasibility.

The discovery of OCPNs is proposed as a divide-and-conquer approach in which each object type defines an individual workflow net [20]. Our mapping and proofs are limited to this characteristic structure of OCPNs.

The discovery technique in [20] has recently been extended in [22], which proposes the discovery of object-centric process trees. Identifier-sound [21] OCPNs can be generated from such process trees, but without synchronization semantics. At the same time, [23] introduces the discovered silent objects that reify and synchronize arbitrary sets of objects. These silent objects may provide insights on how to extend our investigation to many-to-many relationships.

5.3 Conformance Checking and Formal Analysis

OCPNs have been studied for conformance checking. Fitness and precision metrics have been introduced to quantify the suitability of an OCPN with regard to replaying a log [2], and techniques have been studied to calculate alignments for OCPNs [13]. However, both focus only on individual objects and not on their mutual links. For varying links, these approaches are too liberal in identifying deviations [9], calling for richer formalisms such as PNIDs or OPIDs, which are computationally more demanding [9]. Still, with Theorem 2 we show that conformance checking with OCPNs [2, 13] is appropriate if the log satisfies the intended stable relationships. In practice, this check through the discovery of data models [14] can be more performant than the computation of synchronization alignments [9]. If the log violates stable relationships, OPIDs can be used to compute conformance alignments, including specific penalties to quantify the severity of the violations [9]. In the future, the fitness and precision metrics should be extended to respect the object relationships defined in OPIDs.

Other process models have also been considered for object-centric conformance checking. Park et al. use object-centric directly follows graphs [15], which are more simplistic and even more ambiguous than OCPN. On the other hand, Xiu and Li define conformance checking for declarative object-centric behavioral constraints [25], with a consideration of object relationships.

As we show, OPIDs with synchronization can be mapped directly from discovered OCPNs. As OPIDs can emit novel objects, they can serve not only as trace recognizers but also as trace generators. Therefore, our proposed mapping can serve as a missing link between existing works on object-centric process discovery and the formal analysis and execution of such processes. For example, van

[5] `https://anonymous.4open.science/r/To-bind-or-not-to-bind-43DD`

der Werf et al. [21] define correctness notions for Petri nets with identifiers, which can now be applied to discovered process models through our mapping. Finally, OPIDs can also serve as a goal formalism for object-centric process modeling approaches, enabling the comparison of model semantics and discovery results.

Different formal modeling approaches have been proposed for object-centric processes, such as variants of Petri nets with identifiers (PNIDs) [8, 21], or synchronous proclets [5]. These two formalisms provide extended orthogonal features when related to OCPNs. On the one hand, PNIDs provide explicit modeling mechanisms for creating, updating, and removing objects and relationships, while lack the possibility of evolving multiple objects in a single transition firing (cf. variable arcs in OCPNs). On the other hand, proclets extend the variable-arc feature of OCPNs, by supporting different forms of synchronization where multiple/all objects of a certain type synchronously move together with a single, parent object; however, they lack support for the explicit manipulation of objects and relationships, supported in PNIDs as first-class citizen. In a recent effort, *object-centric PNIDs* (OPIDs [9]) have emerged to provide an integrated formalism that unifies the distinctive features of OCPNs with variable arcs, PNIDs and synchronous proclets. Thus, OPID is the natural candidate for enriching an OCPN with the desired synchronization behavior. The relationship between the established approaches of Proclets and PNIDs and the novel OPIDs needs to be studied in the future, where our mapping can now also serve as the missing link for discovering according formalisms.

6 Conclusion

In summary, we show how discovered object-centric process models can be enhanced with explicit data modeling concepts to ensure precision and trust in object relationships. Our contribution rigorously closes the gap between the two representative formal models for object-centric processes: OCPNs—an underfitting model with effective and modular discovery and available conformance checking techniques, and OPIDs—a much more sophisticated model for which discovery techniques are still missing and conformance checking techniques are computationally more demanding. Our findings indicate that when focusing on executions where many-to-one relationships are stable, that is, once a link is established it is rigidly maintained over time, these two models coincide. The implications of this result can be read in two ways. In one direction, it shows that while OCPNs are, in general, underfitting, they become precise enough when describing and prescribing executions that establish stable many-to-one relationships. On the other hand, when restricting attention to this type of executions, discovery and conformance checking for OPIDs can be directly tackled through techniques natively defined for OCPNs. In the future, stability in many-to-many relations in event logs remains to be studied in detail. Furthermore, our results can serve as a promising link between object-centric process discovery and formal analysis, as well as model-driven process execution.

References

1. Adams, J.N., Park, G., van der Aalst, W.M.: ocpa: A python library for object-centric process analysis. Software Impacts **14**, 100438 (2022)
2. Adams, J.N., van der Aalst, W.M.P.: Precision and Fitness in Object-Centric Process Mining. In: Proc. ICPM (2021)
3. Berti, A., et al.: OCEL (Object-Centric Event Log) 2.0 Specification (2024)
4. Carmona, J., van Dongen, B., Solti, A., Weidlich, M.: Conformance Checking: Relating Processes and Models. Springer International Publishing (2018)
5. Fahland, D.: Describing Behavior of Processes with Many-to-Many Interactions. In: Donatelli, S., Haar, S. (eds.) Application and Theory of Petri Nets and Concurrency. pp. 3–24. Springer International Publishing (2019)
6. Fahland, D., et al.: Towards a simple and extensible standard for object-centric event data (OCED) - core model, design space, and lessons learned. CoRR **abs/2410.14495** (2024)
7. Ghahfarokhi, A.F., et al.: OCEL: A Standard for Object-Centric Event Logs. In: New Trends in Database and Information Systems. Springer International Publishing (2021)
8. Ghilardi, S., Gianola, A., Montali, M., Rivkin, A.: Petri net-based object-centric processes with read-only data. Information Systems **107**, 102011 (2022)
9. Gianola, A., Montali, M., Winkler, S.: Object-Centric Conformance Alignments with Synchronization. In: Advanced Information Systems Engineering. Springer Nature Switzerland (2024)
10. Goossens, A., De Smedt, J., Vanthienen, J.: Object-Centric Event Logs: Characteristics, Comparative Analysis and Road Map. In: Business Process Management Forum. Springer Nature Switzerland (2024)
11. Knopp, B., Pourbafrani, M., van der Aalst, W.M.P.: Discovering Object-Centric Process Simulation Models. In: 2023 5th International Conference on Process Mining (ICPM) (2023)
12. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Scalable process discovery and conformance checking. Software & Systems Modeling **17**(2), 599–631 (2018)
13. Liss, L., Adams, J.N., van der Aalst, W.M.P.: Object-centric alignments. In: Proceedings of the 42nd International Conference on Conceptual Modeling - (ER 2023). LNCS, vol. 14320, pp. 201–219. Springer (2023)
14. Liss, L., Adams, J.N., van der Aalst, W.M.P.: TOTeM: Temporal Object Type Model for Object-Centric Process Mining. In: Business Process Management Forum. Springer Nature Switzerland (2024)
15. Park, G., Adams, J.N., van der Aalst, W.M.P.: Conformance Checking and Performance Analysis Using Object-Centric Directly-Follows Graphs. In: Business Process Management Forum. Springer Nature Switzerland (2024)
16. Swevels, A., Fahland, D., Montali, M.: Implementing Object-Centric Event Data Models in Event Knowledge Graphs. In: Process Mining Workshops. Springer Nature Switzerland (2024)
17. van der Aalst, W.M.P.: Object-Centric Process Mining: Dealing with Divergence and Convergence in Event Data. In: Software Engineering and Formal Methods. Springer International Publishing (2019)
18. van der Aalst, W.M.P.: Toward more realistic simulation models using object-centric process mining. In: In Proceedings of the 37th ECMS International Conference on Modelling and Simulation. pp. 5–13. ECMS (2023)

19. van der Aalst, W.M.P., et al.: Process Mining Manifesto. In: Business Process Management Workshops. Springer, Berlin, Heidelberg (2012)
20. van der Aalst, W.M.P., Berti, A.: Discovering Object-centric Petri Nets. Fundamenta Informaticae **175**(1-4) (2020)
21. van der Werf, J.M.E.M., Rivkin, A., Montali, M., Polyvyanyy, A.: Correctness Notions for Petri Nets with Identifiers. Fundamenta Informaticae **190**(2-4) (2024)
22. van Detten, J.N., Schumacher, P., Leemans, S.J.J.: Discovering Compact, Live and Identifier-Sound Object-Centric Process Models. In: 2024 6th International Conference on Process Mining (ICPM) (2024)
23. van Detten, J.N., Schumacher, P., Leemans, S.J.J.: Object Synchronizations and Specializations with Silent Objects in Object-Centric Petri Nets. In: Business Process Management. Springer Nature Switzerland (2024)
24. Weske, M.: Business Process Management: Concepts, Languages, Architectures. Springer Nature (2024)
25. Xiu, B., Li, G.: Diagnosing Conformance Between Object-Centric Event Logs and Models. IEEE Access **11** (2023)
26. Xiu, B., Li, G.: Quantifying Conformance Between Object-Centric Event Logs and Models. IEEE Access **12** (2023)

A Proof for Equivalent OPIDs

To prove the language equivalence of N and $\mathbb{T}_1(N)$ in general, we first define some auxiliary notions.

Given a marking $M\colon Q \to \{0,1\}$ of the OCPN N, we define the corresponding marking $\mathbb{T}_1(M)$ of the OPID $\mathbb{T}_1(N)$ as $\mathbb{T}_1(M)(p) := \{\langle o\rangle \mid o \in \mathcal{O} \wedge M(p,o) > 0\}$.

Given a binding b for an OCPN N, we define a corresponding binding $\beta := \mathbb{T}_1(b)$ for the OPID $\mathbb{T}_1(N)$ as follows: for all $\sigma \in dom(b)$, we set $\beta(v_\sigma) = b(\sigma)$ if $|b(\sigma)| = 1$, and $\beta(V_\sigma) = [b(\sigma)]$ otherwise, where $[b(\sigma)]$ denotes a list containing the elements of the set $b(\sigma)$ in an arbitrary order.

Moreover, we call a sequence of transition firings $M \xrightarrow{*} M'$ in $\mathbb{T}_1(N)$ an *emitting sequence* if it uses only emitting transitions in T_e and a *collapsing sequence* if it uses only consuming transitions in T_c.

We can then show the following equivalence statement:

Theorem 1. *Let O be a set of objects, N be an accepting OCPN, $N' = \mathbb{T}_1(N)$, and $M_\emptyset$ denote the empty marking of N'.*

There exists a run of N:

$$M_0 \xrightarrow{(t_1,b_1)} M_1 \xrightarrow{(t_2,b_2)} \cdots \xrightarrow{(t_n,b_n)} M_n \tag{1}$$

with $M_0{=}M^O_{init}$ and $M_n{=}M^O_{final}$ if and only if there is a run of N' over the set of objects O:

$$M_\emptyset \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset \tag{2}$$

such that $M'_i = \mathbb{T}_1(M_i)$ for all $0 \le i \le n$ and $\beta_i = \mathbb{T}_1(b_i)$ for all $1 \le i \le n$, such that $M_\emptyset \xrightarrow{*} M'_0$ is an emitting sequence, $M'_n \xrightarrow{*} M_\emptyset$ a collapsing sequence, and no emitting or collapsing transitions occur in between.

Proof. (1) First, by definition of T_e in N', there is an emitting sequence $M_\emptyset \xrightarrow{*} M'_0$ since $M'_0 = \mathbb{T}_1(M_0)$ has tokens only in places that are initial in N. Moreover, by definition of T_c there is a collapsing sequence $M'_n \xrightarrow{*} M_\emptyset$ because $M'_n = \mathbb{T}_1(M_n)$ has tokens only in places that are final in N.

Next, we show the following ($\star$): for all $1 \le i \le n$, a transition firing $M_{i-1} \xrightarrow{(t_i,b_i)} M_i$ implies $M'_{i-1} \xrightarrow{(t_i,\beta_i)} M'_i$. Note first that the binding $\beta_i = \mathbb{T}_1(b_i)$ is type-preserving and well-defined since this also holds for b_i. All inscriptions in $\mathbb{T}_1(N)$ are singleton tuples, so the extension $\boldsymbol{\beta}_i$ of β_i to inscriptions is given by $\boldsymbol{\beta}_i(\langle x_\sigma\rangle) = \{\langle \beta_i(x_\sigma)\rangle\}$ for variables $x_\sigma \in \Upsilon \cup X$, and by $\boldsymbol{\beta}_i(\langle X_\sigma\rangle) = \{\langle a\rangle \mid a \text{ occurs in } \beta_i(X_\sigma)\}$ for all $X_\sigma \in X_{list}$. In both cases, we have $\boldsymbol{\beta}_i(x_\sigma) = \boldsymbol{\beta}_i(X_\sigma) = \{\langle a\rangle \mid a \in b_i(\sigma)\}$.

Now, as t_i is enabled with b_i in marking M_{i-1} of N, $cons(t_i,b_i) \le M_{i-1}$ holds for $cons(t_i,b_i) = \{(p,o) \mid p \in \bullet t_i \text{ and } o \in b_i(pt(p))\}$. We have $\boldsymbol{\beta}_i(F_{in}(p,t_i)) = \{\langle o\rangle \mid o \in b_i(pt(p))\}$, and therefore $\boldsymbol{\beta}_i(F_{in}(p,t_i)) \subseteq M'_{i-1}(p)$ for all $p \in \bullet t_i$, so that t_i is enabled in marking $\mathbb{T}_1(M_{i-1})$ of N'. The resulting marking in N is $M_i = (M_{i-1} \setminus cons(t_i,b_i)) \cup prod(t,b)$, for $prod(t,b) = \{(p,o) \mid$

$p \in t\bullet$ and $o \in b(pt(p))\}$. The equivalent marking in N' is given by $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p,t_i))$ for all $p \in \bullet t_i \setminus t_i\bullet$, $M'_i(p) = M'_{i-1}(p) \cup \boldsymbol{\beta}_i(F_{out}(p,t_i))$ for all $p \in t\bullet_i \setminus \bullet t_i$, and $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p,t_i)) \cup \boldsymbol{\beta}_i(F_{out}(p,t_i))$ for all $p \in \bullet t_i \cap t\bullet_i$. We have $\boldsymbol{\beta}_i(F_{in}(p,t_i)) = \boldsymbol{\beta}_i(F_{out}(p,t_i)) = \{\langle o \rangle \mid o \in b_i(pt(p))\}$, so $M'_i = \mathbb{T}_1(M_i)$. Thus $(\star)$ holds.

A simple inductive argument using $(\star)$, in addition to the observations about emitting and collapsing sequences, shows shows that the run (2) is possible.

(2) First, since $M_\emptyset \xrightarrow{*} M'_0$ is an emitting sequence from the initial marking, M'_0 has tokens only in places that are initial in N. So there is a marking M_0 of N that is a valid initial marking of N and satisfies $M'_0 = \mathbb{T}_1(M_0)$.

Next, we show the following $(\star)$: for all $1 \leq i \leq n$, given M_{i-1} which satisfies $M'_{i-1} = \mathbb{T}_1(M_{i-1})$, there are a marking M_i and a binding b_i that satisfy $M_{i-1} \xrightarrow{(t_i,b_i)} M_i$, $M'_i = \mathbb{T}_1(M_i)$, and $\beta_i = \mathbb{T}_1(b_i)$. Since $\mathbb{T}_1$ is injective on markings and bindings, M_i and b_i can actually be obtained from M'_i and β_i by applying the inverse of $\mathbb{T}_1$. As t_i is enabled in M'_{i-1}, it holds that $\boldsymbol{\beta}_i(F_{in}(p,t_i)) \subseteq M'_{i-1}(p)$ for all $p \in \bullet t_i$. Since $\boldsymbol{\beta}_i(F_{in}(p,t_i)) = \{\langle o \rangle \mid o \in b_i(pt(p))\}$ (see Item (1)), $cons(t_i,b_i) \leq M_{i-1}$ holds, so t_i is enabled in M_{i-1}. The resulting marking M'_i in N' satisfies $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p,t_i))$ for all $p \in \bullet t_i \setminus t_i\bullet$, $M'_i(p) = M'_{i-1}(p) \cup \boldsymbol{\beta}_i(F_{out}(p,t_i))$ for all $p \in t\bullet_i \setminus \bullet t_i$, and $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p,t_i)) \cup \boldsymbol{\beta}_i(F_{out}(p,t_i))$ for all $p \in \bullet t_i \cap t\bullet_i$. The resulting marking M_i in N satisfies $M_i = (M_{i-1} \setminus cons(t_i,b_i)) \cup prod(t,b)$. Since $\boldsymbol{\beta}_i(F_{in}(p,t_i)) = \boldsymbol{\beta}_i(F_{out}(p,t_i)) = \{\langle o \rangle \mid o \in b_i(pt(p))\}$, it holds that $M'_i = \mathbb{T}_1(M_i)$. Thus $(\star)$ holds and an inductive argument shows that the run (1) is possible.

Finally, since $M'_n \xrightarrow{*} M_\emptyset$ is a collapsing sequence to an empty marking, $M'_n = \mathbb{T}_1(M_n)$ has tokens only in places that are final in N, so M_{final} is a suitable final marking for N. □

B Proof for Synchronizing OPIDs

In Appendix A, the equivalence of an OCPN N and its OPID representation $N_1 = \mathbb{T}_R(N)$ for replaying an event log is shown. The mapping $\mathbb{T}_R$ implements the stable relationships R into N_1. We prove the correct mapping by showing three things: First, any accepted run in N_2 is also accepted in N_1. Second, any accepted run in N_1, which is conform to the stable relationships R, is accepted by N_2. Third, any accepted run in N_1, which that does not conform to the stable relationships R, is rejected by N_2.

Theorem 2. *Let $N_1 = \mathbb{T}_1(N) = (\Sigma_1, P_1, T_1, F_{in,1}, F_{out,1}, \mathrm{color}_1, \ell_1)$ be the equivalent OPID for the accepting OCPN N over the set of objects $\mathcal{O}$. Let $N_2 = \mathbb{T}_R(N_1) = (\Sigma_2, P_2, T_2, F_{in,2}, F_{out,2}, \mathrm{color}_2, \ell_2)$ for the set R of stable object type relationships, and $M_\emptyset$ denote the empty marking of N_1 and N_2.*
(1) For any accepted run of N_2:

$$M_\emptyset \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset$$

such that $M_\emptyset \xrightarrow{*} M'_0$ *is an emitting and link creation sequence of only transitions in* $t \in T_i \cup T_L \cup T_e$, $M'_n \xrightarrow{*} M_\emptyset$ *is a collapsing sequence of only transitions* $t \in T_c$, *and no transitions* $t \in T_i \cup T_L \cup T_e \cup T_c$ *occur in between, there is an accepted run of* N_1 *of the form*

$$M_\emptyset \xrightarrow{*} M_0 \xrightarrow{(t_1,\beta_1)} M_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M_n \xrightarrow{*} M_\emptyset \tag{3}$$

such that $\forall p \in P_1 \colon M'_i(p) = M_i(p)$ *for all* $0 \leq i \leq n$.

(2) *For an accepted run of* N_1 *that respects the many-to-one relationships* R:

$$M_\emptyset \xrightarrow{*} M_0 \xrightarrow{(t_1,\beta_1)} M_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M_n \xrightarrow{*} M_\emptyset$$

there is an accepted run of N_2 *of the form*

$$M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset \tag{4}$$

such that $\forall p \in P_1 \colon M'_i(p) = M_i(p)$ *for all* $0 \leq i \leq n$.

(3) *For an accepted run of* N_1 *that does not respect the stable relationships* R:

$$M_\emptyset \xrightarrow{*} M_0 \xrightarrow{(t_1,\beta_1)} M_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M_n \xrightarrow{*} M_\emptyset$$

there can be no accepted run of N_2 *of the form*

$$M_\emptyset \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset \tag{5}$$

such that $\forall p \in P_1 \colon M'_i(p) = M_i(p)$ *for all* $0 \leq i \leq n$.

Proof. (1) $M_\emptyset \xrightarrow{*} M'_0$ in N_2 produces a set of objects through T_i, creates links through T_L, and the transitions $t \in T_e$ populate the play places $p \in P_\blacktriangleright$. In M'_0, only the places $P_\blacktriangleright$ and P_L hold tokens, i.e. $M'_0(p) = \emptyset$ for all $p \in P_2 \setminus (P_\blacktriangleright \cup P_L)$, because no transition $t \in T \setminus (T_e \cup T_L$ can remove the remaining tokens from any place $p \in P_b \cup P_a$, as for all those places and transitions it holds that $F_{in}(p,t) = \bot$, and the execution would not be accepted for the final marking $M_\emptyset$.

For N_1, there exists a sequence $M_\emptyset \xrightarrow{*} M_0$ that can generate the same marking for the play places $p \in P_\blacktriangleright \subset P_1$ by firing the appropriate emitting transition for every object. Indeed, for all objects and places $p' \in P_\blacktriangleright \subset P_2$, where $o \in M'_0(p')$ and $ot(o) = \sigma$, there exists an emitting transition $t \in T_e \subset T_1$ with an outflow to the according place $p \in P_\blacktriangleright \subset P_1$ with $\text{color}_2(p) = \sigma$ where $F_{out,1}(t,p) = \langle \nu_\sigma \rangle$. Therefore, the emitting transition can produce o onto p. Hence, given the marking M'_0 in N_2, N_1 can produce an equivalent marking M_0 for all places $p \in P_1$.

Next we show the following ($\star$): For all $1 \leq i \leq n$, a transition firing $M'_{i-1} \xrightarrow{(t_i,\beta'_i)} M'_i$ implies the existence of a firing $M_{i-1} \xrightarrow{(t_i,\beta_i)} M_i$ where for all $p \in P_1$ it holds that $M_{i-1}(p) = M'_{i-1}(p)$. For all transitions $t' \in T_1 \subset T_2$ of N_2 and $t \in T_1$ of N_1 and places $p' \in P_1 \subset P_2$ of N_2 and $p \in P_1$ of N_1, the inflow and outflow are the same for both nets $F_{in,1}(t,p) = F_{in,2}(t',p')$ and

$F_{out,1}(t,p) = F_{out,2}(t',p')$. Hence if the transition (t_i, β_i) is enabled in marking M'_{i-1} of N_2 then it is is also enabled in M_{i-1} of N_1, and the resulting markings satisfy $M_i(p) = M'_i(p)$ for all places $p \in P_1$.
In M'_n, only the places $P_\blacksquare$ and P_L hold tokens, so $M'_n(p) = \emptyset$ for all $p \in P_2 \setminus (P_\blacksquare \cup P_L)$. Otherwise, no transition $t \in T_c \subset T_2$ can remove the remaining tokens from any place p, as for all those places and transitions it holds that $F_{in}(p,t) = \bot$, and the execution would not be accepted for the final marking $M_\emptyset$. Consider the collapsing sequence $M'_n \xrightarrow{*} M_\emptyset$ in N_2 of only transitions $t' \in T_c$ and a marking M_n in N_1, where $M_n(p) = M'_n(p)$ for all places $p \in P_1$. There exists a collapsing sequence in N_1 consisting of only consuming transitions $t \in T_c$, resulting in $M_\emptyset$ as $F_{in,1}(p,t) = \langle x_{\mathrm{color}_2(p)} \rangle$ and $F_{out,1}(t,p') = \bot$ for any $p' \in P_1$.
A simple inductive argument using $(\star)$, and the observations about the emitting, link creating, and the collapsing sequences, show that the run (3) is possible.

(2) For the emitting sequence $M_\emptyset \xrightarrow{*} M_0$ in N_1 and the set of relations R, there is an emitting sequence $M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0$ in N_2 that sets the links correctly, as follows. Let $S \subseteq \Sigma$ be the set of object types that occur in R.
   - The sequence $M_\emptyset \xrightarrow{*} M'_b$ creates objects whose type occurs in R. To that end, it uses transitions $t_i \in T_i$ to obtain a marking M'_b where for all places $p_b \in P_b$ with $\mathrm{color}_2(p_b) = \langle \sigma \rangle$ for $\sigma \in S$, and the unique place $p \in P_\blacktriangleright$ such that $\mathrm{color}_2(p_b) = \mathrm{color}_2(p)$, it holds that $M_b(p_b) = M_0(p)$ because $F_{out,2}(t_i, p_b) = \langle \nu_\sigma \rangle$.
   - The sequence $M'_b \xrightarrow{*} M'_a$ uses only transitions $t_{\sigma_m,\sigma_o} \in T_L$, with the aim to populate P_L with the required links. For every relationship $(\sigma_m, \sigma_o) \in R$, there exist two places $p_{\sigma_m,(\sigma_m,\sigma_o)}, p_{\sigma_o,(\sigma_m,\sigma_o)} \in P_b$, one transition $t_{(\sigma_m,\sigma_o)} \in T_L$, two places $p_{(\sigma_m,\sigma_o),\sigma_m}, p_{(\sigma_m,\sigma_o),\sigma_o} \in P_a$, and one place $p_{(\sigma_m,\sigma_o)} \in P_L$ that satisfy $F_{in,2}(p_{\sigma_m,(\sigma_m,\sigma_o)}, t_{(\sigma_m,\sigma_o)}) = F_{out,2}(t_{(\sigma_m,\sigma_o)}, p_{\sigma_m,(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m} \rangle$ (resp. $F_{in,2}(p_{\sigma_o,(\sigma_m,\sigma_o)}, t_{(\sigma_m,\sigma_o)}) = F_{out,2}(t_{(\sigma_m,\sigma_o)}, p_{\sigma_o,(\sigma_m,\sigma_o)}) = \langle x_{\sigma_o} \rangle$), as well as $F_{out,2}(t_{\sigma_m,\sigma_o}, p_{(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m}, x_{\sigma_o} \rangle$. For every object o, if $ot(o) = \sigma_m$ (resp. $ot(o) = \sigma_o$), then $\langle o \rangle \in M'_b(p_{\sigma_m,(\sigma_m,\sigma_o)})$ (resp. $\langle o \rangle \in M'_b(p_{\sigma_m,(\sigma_m,\sigma_o)})$).
     For every transition $t_{\sigma_m,\sigma_o} \in T_L$ and every object $o_o \in \mathcal{O}$ with $ot(o_o) = \sigma_o$, a binding β_{o_o,σ_m} can be selected that creates all links of this type that are required later. Let O_m be the set of all objects of type σ_m that occur with o_o in the run of N_1, i.e., all objects $o_m \in \mathcal{O}$ such that $ot(o_m) = \sigma_m$ and $\{o_o, o_m\} \subseteq codom(\beta_i)$. Then $\beta_{o_o,\sigma_m}(x_{\sigma_o}) = o_o$ and $\beta_{o_o,\sigma_m}(X_{\sigma_m}) = [O_m]$. As all required objects are present in the according places of P_b, $(t_{\sigma_m,\sigma_o}, \beta_{o_o,\sigma_m})$ can fire. Every object o_o and $o_m \in O_m$ is only considered once by a transition t_{σ_m,σ_o}, as $F_{out,2}(t_{\sigma_m,\sigma_o}, p_{\sigma_m,(\sigma_m,\sigma_o)}) = F_{out,2}(t_{\sigma_m,\sigma_o}, p_{\sigma_o,(\sigma_m,\sigma_o)}) = \bot$. Because the run in N_1 satisfies the relationship $(\sigma_m, \sigma_o) \in R$, for all bindings β_i it holds that if $o_m \in \beta_i(x_{\sigma_m})$ and an object $o'_o \in \beta_i(x_{\sigma_m})$ of type $ot(o_o) = \sigma_o$, then $o'_o = o_o$. Therefore, for all $p_{(\sigma_m,\sigma_o)} \in P_L$ it holds that

for every pair of links $(o_m, o_o), (o'_m, o'_o) \in M'_a(p_{(\sigma_m,\sigma_o)})$ if $o_m = o'_m$ then $o_o = o'_o$. All links in the places P_L are set correctly for the run in N_1.
For all transitions and bindings (t_i, β_i) in the run, the marking of all $p_{(\sigma_m,\sigma_o)} \in P_L$ is not changed as $F_{in,2}(p_{(\sigma_m,\sigma_o)}, t) = F_{out,2}(t, p_{(\sigma_m,\sigma_o)})$ for all $t \in T \setminus (T_L \cup T_c)$. All relationships are stable.

– The sequence $M'_a \xrightarrow{*} M'_0$ fires transitions to populate $P_{\blacktriangleright}$ for every object o in the run: Let $ot(o) = \sigma$. First, if $\sigma \notin S$, there is a flow from the according $t_\sigma \in T_e$ to a place $p \in P_{\blacktriangleright}$ with $F_{out,2}(t_\sigma, p) = \langle \nu_\sigma \rangle$. Second, if $ot(o) \in S$, there is a place $p_a \in P_a$ with $\text{color}_2(p_a) = \langle \sigma \rangle$ that holds a token $\langle o \rangle$. Hence, a binding $\beta(x_\sigma) = o$ exists for all $F_{in,2}(p_a, t_e) = \langle x_\sigma \rangle$, resulting in a marking where $\langle o \rangle \in M'_0(p)$ because $F_{out,2}(t_e, p) = \langle x_\sigma \rangle$.

Next we show the following $(\star)$: For all $1 \leq i \leq n$, a transition firing $M_{i-1} \xrightarrow{(t_i,\beta_i)} M_i$ where for all $p \in P_1$ it holds that $M_{i-1}(p) = M'_{i-1}(p)$, implies the existence of a firing $M'_{i-1} \xrightarrow{(t_i,\beta_i)} M'_i$, such that that $M_i(p) = M'_i(p)$. For all transitions $t \in T_1 \setminus (T_e \cup T_c) = T_2 \setminus (T_i \cup T_L \cup T_e \cup T_c)$ and all places $p \in P_2 \setminus (P_L \cup P_b \cup P_a) = P_1$, it holds that all arc inscriptions are singleton tuples such that $F_{out,2}(t, p) = F_{out,1}(t, p)$, $F_{in,2}(p, t) = F_{in,1}(p, t)$. If the binding β_i considers objects $o_m \in O_m$ with $ot(o_m) = \sigma_m$ and o_o with $ot(o_o) = \sigma_o$ of two types that are in a relationship $(\sigma_m, \sigma_o) \in R$, there is a place $p_{\sigma_m,\sigma_o} \in P_L \subset P_2$ where $O_m \times \{o_o\} \subseteq M'_0(p_{\sigma_m,\sigma_o})$. Because the relationship is stable, we also have $O_m \times \{o_o\} \subseteq M'_{i-1}(p_{\sigma_m,\sigma_o})$. Therefore, for the binding β_i from the run of N_1 can also be used in the run of N_2. As inflow and outflow are equal for all t_i and places $p \in (P_1 \cup P_L) \subset P_2$, for the resulting marking satisfies $M_i(p) = M'_i(p)$.
In M_n, only the places $P_{\blacksquare}$ hold tokens, such that $M_n(p) = \emptyset$ for all $p \in P_1 \setminus P_{\blacksquare}$. Otherwise, no transition $t \in T_c \subset T_1$ could remove the remaining tokens from any place $p \in P_1 \setminus P_{\blacksquare}$, because $F_{in}(p, t) = \perp$, and $M_\emptyset$ could not be reached. For the collapsing sequence $M_n \xrightarrow{*} M_\emptyset$, there exists an equal collapsing sequence $M'_n \xrightarrow{*} M_\emptyset$ of transitions $t \in T_c \subset T_2$. For every object $o_m \in \mathcal{O}$ of type $ot(o_m) = \sigma_m$ and every relationship (σ_m, σ_o), there exists a place $p_{\sigma_m,\sigma_o} \in P_L$ with exactly one token $(o_m, o_o) \in M'_n(p_{\sigma_m,\sigma_o})$. There exists a final place $p \in P_{\blacksquare}$ with $o_m \in M'_n(p)$, and $F_{in,2}(p, t_e) = \langle x_{\sigma_m} \rangle$ and the link place with $F_{in,2}(p_{\sigma_m,\sigma_o}, t_e) = \langle x_{\sigma_m}, x_{\sigma_o} \rangle$. Hence, a binding $\beta'(\langle x_{\sigma_m}, x_{\sigma_o} \rangle) = \{o_m, o_o\}$ exists for every o_m and is enabled. Therefore, all objects can be removed with their many-to-one links.
A simple inductive argument using $(\star)$, and the observations about the emitting and link creating run, and the collapsing run, shows shows that the total run (4) is possible.

(3) A run in N_1 is not conform to a stable relationship $(\sigma_m, \sigma_o) \in R$, if there exist objects $o_m, o_o \in \mathcal{O}$, where $ot(o_m) = \sigma_m$ and $ot(o_o) = \sigma_o$, that contradict the condition $|lo(o_m, \sigma_o)| = 1$ and $lo(o_o, \sigma_m) \neq \emptyset$. A contradiction for N_1 is posed by any of the following three options: (i) $lo(o_m, \sigma_o) = \emptyset$, (ii) $|lo(o_m, \sigma_o)| > 1$, or (iii) $lo(o_o, \sigma_m) = \emptyset$.
All transitions $t \in T_1 \subset T_2$ that operate both object types σ_m and σ_o, also read from the link place for (σ_m, σ_o) as $F_{in,2}(p_{\sigma_m,\sigma_o}, t) = F_{out,2}(t, p_{\sigma_m,\sigma_o}) =$

$\langle X_{\sigma_m}, x_{\sigma_o} \rangle$. Any binding β_i for the transition t operating on σ_m and σ_o is bound to the link place p_{σ_m,σ_o}. A binding β_i including o_m and o_o is only enabled in the marking M'_i if $\langle o_m, o_o \rangle \in M'_i(p_{\sigma_m,\sigma_o})$. $lo()$ is defined on the bindings β_i of the run $M'_0 \xrightarrow{*} M'_n$.

Therefore, we show that the links for (σ_m, σ_o) can not be constructed in any way such that any of the contradictions (i-iii) are possible to hold in an accepted run of N_2. Note that every object o_m and o_o is considered exactly once for a transition that creates links for the relationship (σ_m, σ_o) in N_2 and during the run $M'_0 \xrightarrow{*} M'_n$, all links in p_{σ_m,σ_o} are stable (cf. item (2)). Hence, the following arguments hold true:

(i) To satisfy $lo(o_m, \sigma_o) = \emptyset$, there must exist no binding that operates on o_m and any o_o of $ot(o_o) = \sigma_o$. For any accepted run, o_m must participate in the link creation and there exists an object o_o such that p_{σ_m,σ_o} holds $\langle o_m, o_o \rangle$. A run in N_2 can only be accepted if any binding over o_m and an object of type σ_o binds o_m to the appropriate o_o. Hence, $lo(o_m, \sigma_o) \neq \emptyset$

(ii) To satisfy $|lo(o_m, \sigma_o)| > 1$, bindings must exist that correlate o_m to two objects of type σ_o. However, during link creation, any o_m can be linked to only one object of the type $ot(o_o) = \sigma_o$ such that in any marking only one link exists $|\{\langle o'_m, o'_o \rangle \in M'_i(p_{\sigma_m,\sigma_o}) \mid o_m = o'_m\}| = 1$. Any two bindings that consider o_m and any object o_o must read from the place p_{σ_m,σ_o} and bind objects according to the links. Therefore, both must bind the same o_o. Hence, $|lo(o_m, \sigma_o)| \leq 1$.

(iii) Analog to item (i), $lo(o_o, \sigma_m) = \emptyset$ is not possible as o_o must participate in the link creation to be in an accepted run. As all bindings over o_o and any objects of type σ_m must adhere to the links. Hence $lo(o_o, \sigma_m) \neq \emptyset$

By showing that all three contradictions must be false, the theorem holds true. No run 5 exists for N_2 that creates links for a non-conform execution. □